

GreenTrack Logistics Control Tower

Enterprise Data-Driven Supply Chain
& Carbon Optimization Platform

Prepared by:

Premanathan Aarthi Manivannan

Jeyanth Shanmugasundaram

Rahul Suresh

August 2026

GreenTrack Logistics Control Tower

Enterprise Data-Driven Supply Chain & Carbon Optimization Platform

Table of Contents

1. DESCRIPTION

- 1.1 Participants
- 1.2 What Problem Should Be Solved
- 1.3 Software Functionalities

2. USER STORIES

- 2.1 Profiles and Roles
- 2.2 Dashboard & Analytics
- 2.3 Fleet Management
- 2.4 Order & Route Planning
- 2.5 Alert & Recommendation Engine
- 2.6 ML Prescriptive Simulator
- 2.7 Interactive Map Visualization
- 2.8 Profile & Settings

3. PLANNING AND DEVELOPMENT

- 3.1 Estimation
- 3.2 Milestones

4. FINAL PRODUCT

- 4.1 Installation and Configuration
- 4.2 First Test
- 4.3 Database Management
- 4.4 User Guide

5. STRUCTURE

- 5.1 Architecture
- 5.2 Data Pipeline
- 5.3 ML Pipeline

6. REVIEW

- 6.1 Work Distribution
- 6.2 Time Distribution
- 6.3 Achieved and Unachieved User Stories
- 6.4 Problems

- 6.5 Future Improvements
- 6.6 Software Used

1. DESCRIPTION

1.1 Participants

- Premanathan Aarthi Manivannan
- Jeyanth Shanmugasundaram
- Rahul Suresh

1.2 What Problem Should Be Solved

In modern supply chains, organizations struggle with two critical inefficiencies: **Fragmented Data Visibility** and **High Carbon Footprints**. Dispatchers often rely on outdated systems that send heavy, underutilized combustion-engine vehicles on routes better suited for light electric vans.

A software platform will be developed to provide a centralized, data-driven **Logistics Control Tower** that ingests raw operational freight data, models precise CO2 emission profiles, and applies predictive machine learning algorithms to replace guesswork with **Data-Driven Prescriptive Optimization**.

1.3 Software Functionalities

The platform provides a full-stack dashboard that allows logistics managers to:

- **Monitor real-time KPIs:** total CO2 emissions, distance traveled, orders fulfilled, and estimated carbon savings
- **Manage fleet vehicles:** view all vehicles by type (electric, combustion, hybrid), track utilization, and filter by status
- **Plan and optimize orders:** browse freight orders, view geographic routes on an interactive map, and compare planning scenarios
- **Receive ML-generated alerts:** get prioritized recommendations for load consolidation, vehicle replacement, and route merging
- **Run what-if simulations:** test alternative vehicle assignments using Machine Learning to project CO2 savings and utilization improvements
- **Visualize routes geospatially:** display all delivery routes on an interactive Leaflet map with utilization-coded polylines and stop markers

The system includes a complete data pipeline: CSV data ingestion, PostgreSQL relational modeling, analytics fact table construction, Random Forest ML model training, and a FastAPI REST API serving predictions to a Next.js React frontend.

2. USER STORIES

2.1 Profiles and Roles

2.1.1 FUNC-ACC-010 -- User Roles

Every person using the system receives one of the following roles:

- **ADMIN** -- Full access to all dashboard features, fleet management, order planning, simulation, and alerts
- **DISPATCHER** -- Can view dashboards, manage orders, run simulations, and acknowledge alerts
- **VIEWER** -- Read-only access to dashboards and reports

2.1.2 FUNC-ACC-020 -- Roles and Permissions

Owners of the "ADMIN" role may manage fleet vehicles, configure system settings, and access all analytics. Owners of the "DISPATCHER" role may manage orders and run simulations. Owners of the "VIEWER" role may only view dashboards and reports.

2.2 Dashboard & Analytics

2.2.1 FUNC-DASH-010 -- Real-Time KPI Dashboard

As a logistics manager, I want to view real-time Key Performance Indicators (CO2 emissions, distance traveled, orders fulfilled, cost savings) so that I can monitor fleet performance at a glance.

2.2.2 FUNC-DASH-020 -- Fleet Overview

As a logistics manager, I want to see a fleet overview showing vehicle counts by type, utilization percentages, and electric vs. combustion breakdown so that I can assess fleet health.

2.2.3 FUNC-DASH-030 -- Quick Stats Sidebar

As a logistics manager, I want a quick stats panel showing total vehicles, active routes, pending orders, and an eco score so that I have immediate access to critical metrics.

2.2.4 FUNC-DASH-040 -- Auto-Refresh Data

As a logistics manager, I want the dashboard to auto-refresh every 30 seconds so that I always see current data without manual intervention.

2.3 Fleet Management

2.3.1 FUNC-FLEET-010 -- Vehicle Listing

As a fleet manager, I want to view all vehicles in a searchable grid with ID, type, tonnage, fuel type, emissions, battery level (for EVs), location, and status so that I can manage the fleet efficiently.

2.3.2 FUNC-FLEET-020 -- Vehicle Filtering

As a fleet manager, I want to filter vehicles by type, fuel type, and status so that I can quickly find specific vehicle categories.

2.3.3 FUNC-FLEET-030 -- Vehicle Type Statistics

As a fleet manager, I want to see per-type statistics including vehicle count, average load ratio, and emission intensity (kg CO2/km) so that I can compare vehicle type performance.

2.3.4 FUNC-FLEET-040 -- Electric vs Combustion Breakdown

As a fleet manager, I want to see the count of electric vs combustion vehicles so that I can track the fleet's electrification progress.

2.4 Order & Route Planning

2.4.1 FUNC-ORDER-010 -- Order Listing

As a dispatcher, I want to view all freight orders with distance, CO2 emissions, load ratio, and assigned vehicle so that I can manage deliveries.

2.4.2 FUNC-ORDER-020 -- Order Detail Modal

As a dispatcher, I want to click on an order to see a detailed modal with KPIs (emissions, distance, load ratio, assigned vehicle) and geographic stops so that I can inspect individual orders.

2.4.3 FUNC-ORDER-030 -- Order Filtering

As a dispatcher, I want to filter orders by status (All, Pending, Assigned, Completed) and by route health (Optimal, Moderate, Low) so that I can focus on specific order categories.

2.4.4 FUNC-ORDER-040 -- Scenario Comparison

As a dispatcher, I want to compare planning scenarios (Normal vs Eco Planning) showing duration, emissions, cost, and efficiency so that I can choose the optimal strategy.

2.5 Alert & Recommendation Engine

2.5.1 FUNC-ALERT-010 -- ML-Generated Recommendations

As a logistics manager, I want to receive ML-generated alerts for load consolidation, vehicle replacement, and route merging so that I can optimize operations proactively.

2.5.2 FUNC-ALERT-020 -- Priority Scoring

As a logistics manager, I want each alert to have a priority score (High, Medium, Low) so that I can address the most impactful issues first.

2.5.3 FUNC-ALERT-030 -- Alert Summary Dashboard

As a logistics manager, I want a summary view showing counts of high-priority, sustainability, and optimization alerts so that I can gauge overall fleet optimization status.

2.5.4 FUNC-ALERT-040 -- Actionable Alerts

As a logistics manager, I want to be able to take action on alerts (e.g., navigate to order, dismiss) so that I can act on recommendations directly.

2.6 ML Prescriptive Simulator

2.6.1 FUNC-SIM-010 -- What-If Vehicle Simulation

As a dispatcher, I want to select an order and test alternative vehicle types to project CO2 savings and utilization changes using Machine Learning so that I can make data-driven vehicle assignment decisions.

2.6.2 FUNC-SIM-020 -- Dynamic Vehicle Type Selection

As a dispatcher, I want the simulator to show all available vehicle types with their descriptions, capacity, and fuel type so that I can choose the best alternative.

2.6.3 FUNC-SIM-030 -- Side-by-Side Comparison

As a dispatcher, I want to see a side-by-side comparison of current vs simulated emissions and utilization so that I can visually assess the impact of changing vehicles.

2.6.4 FUNC-SIM-040 -- CO2 Impact Visualization

As a dispatcher, I want a visual bar chart comparing current vs simulated CO2 emissions so that I can quickly understand the environmental impact.

2.6.5 FUNC-SIM-050 -- Open in Full Simulator

As a dispatcher, I want to open an order from the details modal directly in the full simulator panel so that I can seamlessly transition from inspection to simulation.

2.7 Interactive Map Visualization

2.7.1 FUNC-MAP-010 -- Route Map Display

As a dispatcher, I want to view all delivery routes on an interactive Leaflet map with polylines and stop markers so that I can visualize the geographic distribution of orders.

2.7.2 FUNC-MAP-020 -- Utilization-Coded Routes

As a dispatcher, I want routes to be color-coded by utilization (green for optimal, amber for moderate, red for low) with dashed lines for empty miles so that I can identify inefficient routes at a glance.

2.7.3 FUNC-MAP-030 -- Route Selection and Filtering

As a dispatcher, I want to click on a route to highlight it and filter the orders table so that I can focus on specific routes.

2.7.4 FUNC-MAP-040 -- Stop Markers with Details

As a dispatcher, I want numbered stop markers (green for origin, red for destination) with popup details (city, coordinates, utilization) so that I can inspect individual stops.

2.7.5 FUNC-MAP-050 -- Route Health Sidebar

As a dispatcher, I want a clickable sidebar showing route health counts (Optimal, Moderate, Low) that filters both the map and orders table so that I can quickly focus on problematic routes.

2.8 Profile & Settings

2.8.1 FUNC-PROF-010 -- User Profile Display

As a user, I want to view my profile information (name, email, phone, role, department, access level) so that I can verify my account details.

2.8.2 FUNC-PROF-020 -- Notification Preferences

As a user, I want to configure notification preferences (email alerts, push notifications, weekly reports) so that I control how I receive updates.

2.8.3 FUNC-PROF-030 -- Report Generation

As a user, I want to generate Weekly, Monthly, and Annual reports so that I can export fleet performance data.

3. PLANNING AND DEVELOPMENT

3.1 Estimation

User Story	Estimated Effort
FUNC-ACC-010 -- Define and manage user roles	6 hours
FUNC-ACC-020 -- Enforce permissions based on roles	5 hours
FUNC-DASH-010 -- Real-time KPI dashboard	10 hours
FUNC-DASH-020 -- Fleet overview panel	6 hours
FUNC-DASH-030 -- Quick stats sidebar	4 hours
FUNC-DASH-040 -- Auto-refresh data	3 hours
FUNC-FLEET-010 -- Vehicle listing with search	8 hours
FUNC-FLEET-020 -- Vehicle filtering	5 hours
FUNC-FLEET-030 -- Vehicle type statistics	6 hours
FUNC-FLEET-040 -- Electric vs combustion breakdown	4 hours
FUNC-ORDER-010 -- Order listing	8 hours
FUNC-ORDER-020 -- Order detail modal	7 hours
FUNC-ORDER-030 -- Order filtering (status + health)	6 hours
FUNC-ORDER-040 -- Scenario comparison	5 hours
FUNC-ALERT-010 -- ML-generated recommendations	12 hours
FUNC-ALERT-020 -- Priority scoring	5 hours
FUNC-ALERT-030 -- Alert summary dashboard	4 hours
FUNC-ALERT-040 -- Actionable alerts	4 hours
FUNC-SIM-010 -- What-if vehicle simulation	14 hours
FUNC-SIM-020 -- Dynamic vehicle type selection	5 hours
FUNC-SIM-030 -- Side-by-side comparison	6 hours
FUNC-SIM-040 -- CO2 impact visualization	5 hours

User Story	Estimated Effort
FUNC-SIM-050 -- Open in full simulator	4 hours
FUNC-MAP-010 -- Route map display	10 hours
FUNC-MAP-020 -- Utilization-coded routes	6 hours
FUNC-MAP-030 -- Route selection and filtering	7 hours
FUNC-MAP-040 -- Stop markers with details	5 hours
FUNC-MAP-050 -- Route health sidebar	5 hours
FUNC-PROF-010 -- User profile display	3 hours
FUNC-PROF-020 -- Notification preferences	3 hours
FUNC-PROF-030 -- Report generation	4 hours
Data Pipeline (ETL + DB)	20 hours
ML Training Pipeline	15 hours
Backend API Development	18 hours
Frontend Integration	12 hours
TOTAL	~227 hours

3.2 Milestones

MSid	Description	User Stories
MS1	Data Pipeline & Database -- CSV data ingestion, PostgreSQL schema design, ETL pipeline, and fact table construction are implemented. The foundational data layer is complete.	Data ingestion, DB schema, ETL, fact builder
MS2	Backend API -- FastAPI REST endpoints for dashboard, fleet, orders, routes, alerts, and simulation are implemented and tested.	FUNC-DASH-010, FUNC-FLEET-010/030/040, FUNC-ORDER-010, FUNC-MAP-010, FUNC-ALERT-010/020, FUNC-SIM-010
MS3	ML Pipeline -- Feature engineering, model training (emission + load), inference, and simulation engine are implemented.	FUNC-SIM-010, FUNC-SIM-020, FUNC-ALERT-010

MSid	Description	User Stories
MS4	Frontend Dashboard & Fleet -- React dashboard with KPIs, fleet view with vehicle cards, profile view, and mobile navigation are implemented.	FUNC-DASH-010/020/030/040, FUNC-FLEET-010/020/030/040, FUNC-PROF-010/020/030
MS5	Frontend Orders & Map -- Orders view, interactive Leaflet map with route visualization, order detail modal, scenario comparison, and route health filtering are implemented.	FUNC-ORDER-010/020/030/040, FUNC-MAP-010/020/030/040/050
MS6	Frontend Alerts & Simulator -- Alerts view with recommendation engine integration, ML Prescriptive Simulator panel with side-by-side comparison and CO2 visualization are implemented.	FUNC-ALERT-010/020/030/040, FUNC-SIM-010/020/030/040/050

4. FINAL PRODUCT

4.1 Installation and Configuration

4.1.1 Prerequisites

The following software must be installed on your system before setting up GreenTrack:

- **Python 3.11+**
 - Download from: <https://www.python.org/>
 - Required for backend, ML pipeline, and data processing
- **PostgreSQL 14+**
 - Download from: <https://www.postgresql.org/>
 - Required for data persistence
- **Node.js 20+**
 - Download from: <https://nodejs.org/>
 - Required for the Next.js frontend
- **Git**
 - Download from: <https://git-scm.com/>
 - Required for cloning the repository

4.1.2 Source Repository

The Git repository can be found at the Makeathon project directory.

4.1.3 Installation

Step 1: Clone the Repository

```
git clone <repository-url>
cd Makeathon
```

Step 2: Set Up Python Environment

```
python -m venv venv
venv\Scripts\activate # Windows
pip install -r requirements.txt
```

Step 3: Configure Database

Copy `.env.example` to `.env` and update the database credentials:

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=greentrack
```

```
DB_USER=greentrack_user
DB_PASSWORD=your_password_here
```

Step 4: Initialize Database and Load Data

```
python main.py init-db
python main.py ingest
python main.py build-facts
```

Step 5: Train ML Models

```
python main.py train-models
```

Step 6: Start Backend API

```
uvicorn app.api.main:app --reload --port 8000
```

Step 7: Set Up Frontend

```
cd frontend
npm install
npm run dev
```

The frontend will be available at <http://localhost:3000>.

4.2 First Test

4.2.1 Testing the Backend API

Open a web browser and navigate to:

```
http://127.0.0.1:8000/api/health
```

Expected Result:

```
{"status": "ok"}
```

4.2.2 Testing Dashboard Summary

```
GET http://127.0.0.1:8000/api/dashboard/summary
```

Expected Result: JSON with total_co2_emission, total_distance_km, average_load_ratio, number_of_orders, estimated_co2_savings.

4.2.3 Testing Fleet Overview

```
GET http://127.0.0.1:8000/api/fleet/overview
```

Expected Result: JSON with total_vehicles, vehicle_counts_by_type, electric_vehicles, combustion_vehicles, average_utilization.

4.2.4 Testing Simulation

```
python main.py simulate --order 1 --vehicle-type ZFT003
```

Expected Result: CO2 savings percentage and utilization change for the specified order and vehicle type.

4.2.5 Frontend Dashboard

Navigate to <http://localhost:3000> in your browser. You should see the Dashboard view with live KPI data, fleet overview, and quick stats.

4.3 Database Management

4.3.1 PostgreSQL Database

The system uses PostgreSQL with the following tables:

Table	Description
addresses	Physical locations with lat/lon coordinates
transport_types	Vehicle type definitions (ZFT003, ZFT004, ZFT005)
vehicles	Individual vehicle records with license plates
vehicle_attributes	Capacity and emission coefficients per type
freight_units	Unit-level freight data
freight_unit_stops	Stop sequences for freight units
freight_orders	Order-level freight data
freight_order_items	Items within orders
freight_order_stops	Geographic stops for orders
freight_order_stages	Distance/duration between stops
transport_stage_fact	Analytics fact table (flattened star-schema)

4.3.2 CLI Commands

```
python main.py init-db # Create tables
python main.py init-db --reset # Drop and recreate tables
```



```
python main.py ingest # Load CSV data
python main.py build-facts # Build analytics fact table
python main.py analytics-report # Print KPI report
python main.py train-models # Train ML models
python main.py simulate --order ID # Run simulation
```

4.4 User Guide

4.4.1 Dashboard View

The Dashboard displays four KPI cards at the top:

- **CO2 Emissions** -- Total emissions in kg with live indicator
- **Distance Traveled** -- Total distance in km with live indicator
- **Orders Fulfilled** -- Count of processed orders
- **Cost Savings** -- Estimated CO2 savings in kg

Below the KPIs, the **Fleet Overview** shows utilization bars for each vehicle type (Electric vs Combustion). The **Quick Stats** sidebar shows total vehicles, active routes, pending orders, and eco score.

4.4.2 Fleet View

The Fleet view displays all vehicles in a searchable card grid. Each card shows:

- Vehicle ID and type
- Tonnage and fuel type (with icon)
- Emissions (kg CO2)
- Battery level (for electric vehicles)
- Status badge (Active, Idle, Maintenance)

Use the search bar to filter vehicles by ID, type, or status.

4.4.3 Orders & Route Planning

The Orders view contains three main sections:

1. **Live Route Visualization** -- Interactive Leaflet map showing all delivery routes. Click a route to highlight it. Routes are color-coded by utilization.
2. **Scenario Comparison** -- Side-by-side comparison of Normal vs Eco Planning scenarios with duration, emissions, cost, and efficiency metrics.
3. **Customer Orders Table** -- Filterable table with tabs for All/Pending/Assigned/Completed orders. Click "Details" to open the order detail modal. Click "Show on Map" to filter the map to a specific order.

The **Route Health Sidebar** provides clickable filters (Optimal/Moderate/Low) that filter both the map and orders table simultaneously.

4.4.4 Alerts View

The Alerts view shows:

- **Summary Cards** -- High Priority, Sustainability, and Optimizations counts
- **Alert List** -- Scrollable list of ML-generated recommendations with type icons, priority badges, and timestamps
- **Actions** -- Take Action (navigates to related order) and Dismiss buttons

4.4.5 ML Prescriptive Simulator

The Simulator panel (accessible from Orders view or via "Open in Full Simulator" in the order details modal) provides:

1. **Current Order Info** -- Distance, CO2, and load ratio for the selected order
2. **Vehicle Type Selector** -- Dropdown with all vehicle types, descriptions, and capacities
3. **Run Simulation** -- Executes ML prediction and physics-based CO2 calculation
4. **Results** -- Recommendation banner, side-by-side Before/After comparison, CO2 Impact bar chart

4.4.6 Profile View

The Profile view shows user information, notification preferences (toggles for email/push/weekly reports), report generation buttons (Weekly/Monthly/Annual), and access & roles display.

5. STRUCTURE

5.1 Architecture

5.1.1 System Architecture Overview

GreenTrack follows a **three-tier architecture**:

```
+-----+ +-----+ +-----+
| | | | | |
| Next.js React | --> | FastAPI REST | --> | PostgreSQL |
| Frontend (3000) | | API (8000) | | Database |
| | | | |
+-----+ +-----+ +-----+
|
|
v
+-----+
| |
| ML Pipeline |
| (Scikit-Learn) |
| |
+-----+
```

5.1.2 Component Diagram

```
Frontend (Next.js)
|-- Dashboard View (KPIs, Fleet Overview, Quick Stats)
|-- Fleet View (Vehicle Cards, Search/Filter)
|-- Orders View (Orders Table, Route Map, Scenarios, Simulator)
|-- Alerts View (Recommendations, Priority Scoring)
|-- Profile View (Settings, Reports)
|-- Route Map (Leaflet, OpenStreetMap)
|-- Simulation Panel (ML Predictions, Side-by-Side)

Backend (FastAPI)
|-- /api/dashboard/summary
|-- /api/fleet/overview, /api/fleet/vehicle-types
|-- /api/orders, /api/orders/{id}
|-- /api/routes/map
|-- /api/alerts/recommendations
|-- /api/simulate (POST)
|-- /api/health

ML Pipeline
|-- Feature Engineering (features.py)
|-- Model Training (training.py) -- RandomForestRegressor
|-- Inference (inference.py) -- predict()
|-- Simulation (simulator.py) -- simulate_order()
|-- Recommendation Engine (engine.py)

Data Layer
|-- CSV Ingestion (csv_loader.py)
|-- SQLAlchemy ORM (models.py)
|-- Fact Table Builder (fact_builder.py)
|-- Materialized Views
```

5.2 Data Pipeline

5.2.1 Ingestion Flow

```
Raw CSV Files --> csv_loader.py --> PostgreSQL Tables
(ADDRESSDATA.csv) (addresses)
(MEANS_OF_TRANSPORT.csv) (transport_types)
(RESSOURCE_HEAD.csv) (vehicles)
(RESSOURCE_EQUIPMENT.csv) (vehicle_attributes)
(freight_order_*.csv) (freight_orders, items, stops, stages)
```

5.2.2 Analytics Pipeline

```
PostgreSQL Tables --> fact_builder.py --> transport_stage_fact
--> materialized views
(fleet_utilization)
(emissions_per_vehicle)
(emissions_per_order)
```

5.3 ML Pipeline

5.3.1 Feature Engineering

The ML pipeline extracts features from `transport_stage_fact`:

Feature	Description
distance_km	Distance of the transport stage
load_weight	Total weight carried
vehicle_capacity	Maximum vehicle capacity
load_ratio	load_weight / vehicle_capacity
emission_per_km	CO2 / distance ratio
vehicle_type_encoded	Label-encoded vehicle type
weekday	Day of week from created_at

5.3.2 Model Training

Two `RandomForestRegressor` models are trained with `GridSearchCV` hyperparameter tuning:

- **Emission Model** -- Predicts CO2 emissions (kg) based on distance, load, capacity, and vehicle type

- **Load Model** -- Predicts load ratio based on distance, weight, capacity, and emission intensity

Models are persisted as `emission_model.joblib` and `load_model.joblib`.

5.3.3 Simulation Engine

The simulation uses a **hybrid approach**:

1. **CO2 Calculation** -- Physics-based formula using vehicle emission coefficients:

```
CO2 = distance * (co2_empty + load_ratio * (co2_loaded - co2_empty))
```

2. **Load Ratio** -- Direct computation: `total_weight / new_capacity`
3. **Savings** -- Percentage difference between current and alternative CO2 predictions

5.3.4 Recommendation Engine

The recommendation engine (`engine.py`) generates alerts based on:

- **Load Consolidation** -- Orders with load ratio < 40% are candidates for consolidation
- **Vehicle Replacement** -- High emission/km routes suggest switching to electric vehicles
- **Route Merge** -- Orders with distance > 200km and low load are candidates for route merging

Each recommendation receives a priority score based on estimated CO2 reduction potential.

6. REVIEW

6.1 Work Distribution

User Story	Assigned To
FUNC-ACC-010 -- User Roles	Premanathan
FUNC-ACC-020 -- Roles and Permissions	Premanathan
FUNC-DASH-010 -- Real-time KPI Dashboard	Rahul
FUNC-DASH-020 -- Fleet Overview	Rahul
FUNC-DASH-030 -- Quick Stats Sidebar	Jeyanth
FUNC-DASH-040 -- Auto-Refresh	Jeyanth
FUNC-FLEET-010 -- Vehicle Listing	Rahul
FUNC-FLEET-020 -- Vehicle Filtering	Rahul
FUNC-FLEET-030 -- Vehicle Type Statistics	Jeyanth
FUNC-FLEET-040 -- Electric vs Combustion	Jeyanth
FUNC-ORDER-010 -- Order Listing	Rahul
FUNC-ORDER-020 -- Order Detail Modal	Rahul
FUNC-ORDER-030 -- Order Filtering	Rahul
FUNC-ORDER-040 -- Scenario Comparison	Jeyanth
FUNC-ALERT-010 -- ML Recommendations	Premanathan
FUNC-ALERT-020 -- Priority Scoring	Premanathan
FUNC-ALERT-030 -- Alert Summary Dashboard	Premanathan
FUNC-ALERT-040 -- Actionable Alerts	Jeyanth
FUNC-SIM-010 -- What-If Simulation	Premanathan
FUNC-SIM-020 -- Dynamic Vehicle Selection	Premanathan
FUNC-SIM-030 -- Side-by-Side Comparison	Premanathan
FUNC-SIM-040 -- CO2 Impact Visualization	Premanathan
FUNC-SIM-050 -- Open in Full Simulator	Jeyanth
FUNC-MAP-010 -- Route Map Display	Rahul

User Story	Assigned To
FUNC-MAP-020 -- Utilization-Coded Routes	Rahul
FUNC-MAP-030 -- Route Selection/Filtering	Rahul
FUNC-MAP-040 -- Stop Markers	Jeyanth
FUNC-MAP-050 -- Route Health Sidebar	Jeyanth
FUNC-PROF-010 -- User Profile	Jeyanth
FUNC-PROF-020 -- Notification Preferences	Jeyanth
FUNC-PROF-030 -- Report Generation	Jeyanth
Data Pipeline & ETL	Premanathan
ML Training Pipeline	Premanathan
Backend API Development	Premanathan
Frontend Integration	Rahul

6.2 Time Distribution

6.2.1 Premanathan Aarthi Manivannan

Task	Time Spent (Hours)
FUNC-ACC-010 -- User Roles	6
FUNC-ACC-020 -- Roles and Permissions	5
FUNC-ALERT-010 -- ML Recommendations	12
FUNC-ALERT-020 -- Priority Scoring	5
FUNC-ALERT-030 -- Alert Summary	4
FUNC-SIM-010 -- What-If Simulation	14
FUNC-SIM-020 -- Dynamic Vehicle Selection	5
FUNC-SIM-030 -- Side-by-Side Comparison	6
FUNC-SIM-040 -- CO2 Impact Visualization	5
Data Pipeline & ETL	20
ML Training Pipeline	15
Backend API Development	18

Task	Time Spent (Hours)
TOTAL	115

6.2.2 Rahul Suresh

Task	Time Spent (Hours)
FUNC-DASH-010 -- Real-time KPI Dashboard	10
FUNC-DASH-020 -- Fleet Overview	6
FUNC-FLEET-010 -- Vehicle Listing	8
FUNC-FLEET-020 -- Vehicle Filtering	5
FUNC-ORDER-010 -- Order Listing	8
FUNC-ORDER-020 -- Order Detail Modal	7
FUNC-ORDER-030 -- Order Filtering	6
FUNC-MAP-010 -- Route Map Display	10
FUNC-MAP-020 -- Utilization-Coded Routes	6
FUNC-MAP-030 -- Route Selection/Filtering	7
Frontend Integration	12
TOTAL	85

6.2.3 Jeyanth Shanmugasundaram

Task	Time Spent (Hours)
FUNC-DASH-030 -- Quick Stats Sidebar	4
FUNC-DASH-040 -- Auto-Refresh	3
FUNC-FLEET-030 -- Vehicle Type Statistics	6
FUNC-FLEET-040 -- Electric vs Combustion	4
FUNC-ORDER-040 -- Scenario Comparison	5
FUNC-ALERT-040 -- Actionable Alerts	4
FUNC-SIM-050 -- Open in Full Simulator	4
FUNC-MAP-040 -- Stop Markers	5

Task	Time Spent (Hours)
FUNC-MAP-050 -- Route Health Sidebar	5
FUNC-PROF-010 -- User Profile	3
FUNC-PROF-020 -- Notification Preferences	3
FUNC-PROF-030 -- Report Generation	4
TOTAL	50

6.3 Achieved and Unachieved User Stories

MSid	Description	Status
MS1	Data Pipeline & Database	Fully Achieved. CSV ingestion, PostgreSQL schema, ETL pipeline, and fact table construction are complete.
MS2	Backend API	Fully Achieved. All 9 REST endpoints implemented and tested.
MS3	ML Pipeline	Fully Achieved. Feature engineering, model training, inference, and simulation engine complete.
MS4	Frontend Dashboard & Fleet	Fully Achieved. Dashboard with KPIs, fleet view with vehicle cards, profile view implemented.
MS5	Frontend Orders & Map	Fully Achieved. Orders view, Leaflet map, order detail modal, scenario comparison, route health filtering implemented.
MS6	Frontend Alerts & Simulator	Fully Achieved. Alerts view with ML recommendations, simulation panel with side-by-side comparison and CO2 visualization implemented.

Not Achieved / Partially Implemented:

- Real-time WebSocket notifications (planned but not implemented -- current dashboard uses polling)
- User authentication and role-based access control (UI assumes a logged-in user)
- Historical trend charts (planned but not implemented in current dashboard)

- Export functionality for reports (buttons exist but backend export not implemented)

6.4 Problems

6.4.1 Frontend-Backend API Connection

The frontend initially could not connect to the backend. The root cause was that `localhost:8000` was intercepted by a Docker backend service running on the same port. The fix was to change the API URL in `.env.local` from `localhost` to `127.0.0.1`.

6.4.2 React Hydration Errors

Browser extensions were modifying the HTML after server-side rendering, causing React hydration mismatch errors. The fix was to add `suppressHydrationWarning` to the `<body>` tag in the root layout.

6.4.3 ML Simulation Utilization Always Zero

The ML model's `predicted_load_ratio` was returning similar values for both current and alternative vehicle types, making the utilization change always ~0%. The fix was to compute utilization directly from physics (`total_weight / new_capacity`) instead of relying on the ML model prediction.

6.4.4 Vehicle Type Descriptions Missing

The fleet API only returned vehicle type codes (ZFT003, ZFT004, ZFT005) without human-readable descriptions. The fix was to create a new `/api/fleet/vehicle-types` endpoint that queries the `transport_types` table for descriptions.

6.4.5 Simulation Panel State Not Preserved

When clicking "Open in Full Simulator" from the order details modal, the selected order was cleared when the modal closed. The fix was to decouple modal visibility state from the selected order state.

6.5 Future Improvements

- **Real-Time Updates** -- Replace SWR polling with WebSocket connections for live dashboard updates
- **User Authentication** -- Implement JWT-based authentication with role-based access control
- **Historical Analytics** -- Add trend charts showing CO2 emissions, fleet utilization, and cost savings over time
- **Export Functionality** -- Enable PDF/CSV export for reports and analytics
- **Mobile App** -- Develop a native mobile app for field dispatchers
- **Route Optimization** -- Integrate a route optimization engine (e.g., Google OR-Tools) for automated route planning

- **Multi-Depot Support** -- Extend the system to handle multiple warehouse/depot locations
- **Carbon Offset Integration** -- Calculate and suggest carbon offset programs based on emission data
- **Predictive Maintenance** -- Use vehicle telemetry data to predict maintenance needs
- **API Documentation** -- Add OpenAPI/Swagger documentation for all endpoints

6.6 Software Used

6.6.1 Backend

Technology	Purpose
Python 3.14	Primary backend language
FastAPI	REST API framework
SQLAlchemy	ORM for PostgreSQL
Pydantic	Data validation and serialization
Pandas	Data manipulation and ETL
Scikit-Learn	Machine learning (RandomForestRegressor)
Joblib	Model persistence

6.6.2 Database

Technology	Purpose
PostgreSQL	Primary relational database
psycopg2	PostgreSQL adapter for Python

6.6.3 Frontend

Technology	Purpose
Next.js 16	React framework with SSR
React 19	UI library
TypeScript	Type-safe JavaScript
Tailwind CSS	Utility-first CSS framework
ShadCN UI	Component library (57 components)

Technology	Purpose
Axios	HTTP client
SWR	Data fetching with caching and revalidation
Leaflet / React-Leaflet	Interactive map visualization
Lucide React	Icon library
Vercel Analytics	Usage analytics

6.6.4 Development Tools

Technology	Purpose
Git	Version control
Flake8	Python linter
MyPy	Python type checker
Prettier	Code formatter
Postman	API testing
ESLint	JavaScript linter

Document generated for the GreenTrack Logistics Control Tower project.

August 2026